Statement of Work (SoW) — Project: AutoVaccinate-RAG-Lite

Length: 6–8 weeks, with Weeks 7–8 as stretch
Supervisor: Havva (~~weekly 30-min stand-up each Tuesday, ET~~)
Candidate: PhD applicant (remote)
Objective: Prototype a small RAG system that detects its own factual/regression failures and applies the cheapest effective patch under knowledge-graph (KG) constraints. Patches: retriever params, index refresh, prompt edits, reranker toggle, tiny LoRA adapters.

Scope & components

- Task/data: start with FEVER (fact verification) + a 1–2 domain add-on (e.g., movies via WikiMovies/MetaQA) to keep KG compact; optionally test on a 1k-sample HotpotQA subset for multi-hop (see: https://huggingface.co/papers/trending, https://github.com/yuyuz/MetaQA?utm_source=chatgpt.com, https://huggingface.co/datasets/facebook/wiki_movies?utm_source=chatgpt.com, https://fever.ai/dataset/fever.html?utm_source=chatgpt.com)
- KG source: tiny domain KG (edges from WikiMovies/MetaQA KB; or a Wikidata slice) (see: https://github.com/yuyuz/MetaQA?utm_source=chatgpt.com)
- RAG stack: LlamaIndex or LangChain + FAISS/SQLite/Chroma; sentence-transformers all-MiniLM-L6-v2 for CPU-friendly embeddings (see: https://docs.llamaindex.ai/en/stable/examples/low_level/oss_ingestion_retrieval/?utm_source=chatgpt.com, https://python.langchain.com/docs/tutorials/rag/?utm_source=chatgpt.com)
- Failure detectors:
  - KG-consistency check on entity–relation pairs in answers.
  - Fact-verification signal (FEVER-style retrieval + NLI lite).
  - Optional Self-RAG-style critique token or post-hoc critique to trigger patching (https://arxiv.org/abs/2310.11511?utm_source=chatgpt.com).
- Patch set:
  - Retriever (k, BM25 vs dense, re-index),
  - Prompt template edits,
  - Reranker on/off,
  - LoRA micro-adapter on failure shards (PEFT/LoRA) with strict budget (e.g., 1–2K steps on 8B int4) (see: https://huggingface.co/docs/peft/en/package_reference/lora?utm_source=chatgpt.com)
- Patch selection: contextual bandit (LinUCB/Thompson) optimizing (factuality↑, KG-consistency↑, latency/VRAM↓).
- Evaluation: RAGAS faithfulness/relevancy + task accuracy; report cost/latency deltas per patch type (see: https://docs.ragas.io/en/stable/?utm_source=chatgpt.com)

Weekly Plan

- W1: Stand up minimal RAG on FEVER dev split; implement KG loader (movie KB or small Wikidata slice) (see: fever.ai, Hugging Face)
- W2: Implement detectors: KG-consistency and FEVER-style entailment check; define failure labels (see: ACL Anthology)
- W3: Implement patch primitives (k/ties, prompt edits, reranker) + evaluation harness (RAGAS + task accuracy) (see: Ragas)
- W4: Add contextual-bandit patch selector with cost budget; log decisions and outcomes.
- W5: Add LoRA micro-patch option (strict budget); compare CPU-only vs GPU-assisted paths (see: Hugging Face)
- W6: HotpotQA 1k subset to test multi-hop; run ablations and failure taxonomy (see: Papers with Code)
- W7–8 (stretch): Try a light Self-RAG critique stage or explore RAGBench/THELMA evaluators (see: arXiv+2arXiv+2)

Deliverables

- Code: reproducible RAG with KG checker, bandit patcher, and eval harness; Docker + tests.
- Repro kit: scripts to re-create the KG, index, and patch experiments.
- Mini-paper: 4–6 pages with a formal cost-bounded patch selection section and KG-consistency constraints.
- Ethics note: attribution, license compliance for data/models; safe-handling of generated content.

Success criteria

- ≥5–10 pt gain on FEVER/HotpotQA accuracy or RAGAS faithfulness versus static RAG, while meeting a fixed latency/VRAM budget.
- Evidence that the bandit chooses different patch types by failure mode (retrieval vs generation), not just one blunt fix.